

인공지능시스템

AI System

int**Ro**-Lab.

청주대학교 무인항공기학과
김상현 교수 (www.intro-lab.com)

공지: 중간평가

- ◆ 과제 주제: 인공지능 드론에 대한 아이디어 제안
- ◆ 제출 형식: 파워포인트 10슬라이드 분량 (PPT, PDF 모두 제출)
- ◆ 작성 내용
 - 강의에서 배운 내용 및 인공지능에 대한 전반적인 기술을 반영하여 아이디어를 제시.
- ◆ 발표 내용
 - 1. 드론 시스템 구성 및 임무
 - 2. 적용한 인공지능 기술
 - 3. 기존 드론 시스템과의 차별점.
- ◆ 아이디어 참고 예시: Skydio F10 Drone
- ◆ 제출 기한: 10월 27일 (화) 오후 11시
- ◆ 제출 방법: 에델바이스 제출
- ◆ 발표 일정: 10월 29일 (목) 정규 수업 시간
- ◆ 평가 기준: 발표 내용의 전문성(50%), 다른 학생과의 차별성(50%)

마감 이후 제출: 감점

◆ Introducing Skydio F10 Lightrunner + MegaDock | Autonomous Fixed-Wing Drone



◆ 베이지 정리



◆ 베이즈 정리

● 사전 확률과 사후 확률

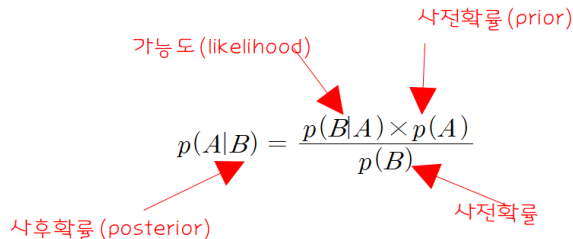
$$p(A|B) = \frac{A \text{와 } B \text{가 동시에 발생할 경우의 수}}{B \text{가 발생할 경우의 수}}$$

$$p(A|B) = \frac{p(A \cap B)}{p(B)}$$

$$p(B|A) = \frac{p(A \cap B)}{p(A)}$$

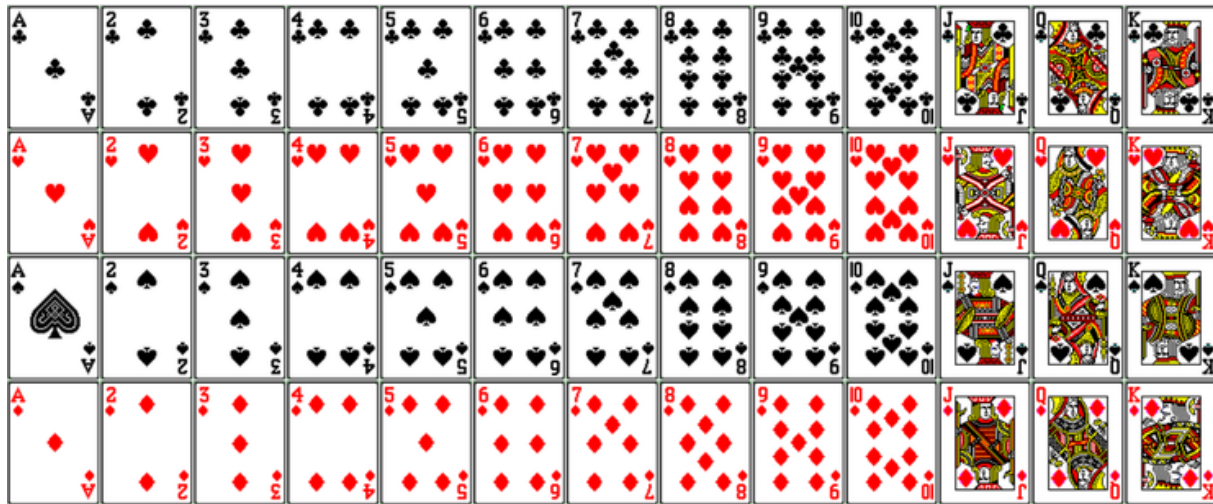
$$p(A \cap B) = p(B|A) \times p(A)$$

$$p(A|B) = \frac{p(B|A) \times p(A)}{p(B)}$$



◆ 베イズ 정리

- 예시: 카드 게임
- 우리가 한 장의 카드를 가지고 있는데 카드에 얼굴이 그려져 있다는 것만 알고 있다. 이 상황에서 이 카드가 킹(king)일 확률을 추론해보자. 즉 $P(\text{King} \mid \text{Face})$ 확률을 계산해보는 것이다.



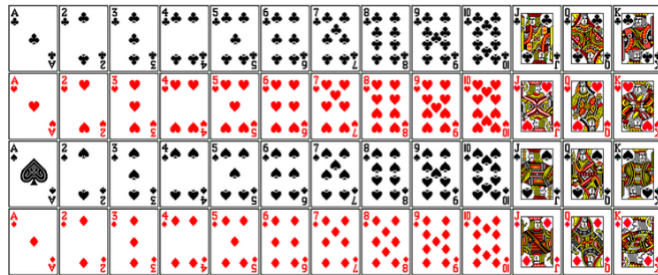
◆ 베이지 정리

● 예시: 카드 게임

$$p(A|B) = \frac{p(B|A) \times p(A)}{p(B)}$$

$$P(King|Face) = \frac{P(Face|King)P(King)}{P(Face)}$$

$$= \frac{1 \times \frac{1}{13}}{\frac{3}{13}} = \frac{1}{3}$$



◆ 베이즈 정리

● 예시: 충치

➤ 치통(B)이 있을 때, 충치(A)일 확률

	치통	¬치통
충치	0.04	0.06
¬충치	0.01	0.89

$$p(A|B) = \frac{p(B|A) \times p(A)}{p(B)}$$

$$P(\text{충치} | \text{치통}) = \frac{P(\text{충치} \wedge \text{치통})}{P(\text{치통})} = \frac{0.04}{(0.04 + 0.01)} = 0.8$$

◆ 베이즈 정리

● 몬티홀 문제와 베이즈 정리

1. '선택 유지' 전략의 베이즈 정리

- 가설 (H_{keep}): 처음 선택을 유지하는 것이 정답이다 (즉, 차가 A에 있다).
- 증거 (E): 사회자가 B 문을 열어 염소를 보여주었다.

1) 사전 확률 $P(H_{keep})$

처음에 차가 A에 있을 확률이므로 $\frac{1}{3}$ 입니다.

2) 우도 $P(E|H_{keep})$

만약 유지하는 것이 정답(차가 A에 있음)이라면, 사회자는 남은 B와 C 중 아무 문이나 열 수 있으므로 B를 열 확률은 $\frac{1}{2}$ 입니다.

3) 사후 확률 계산

사회자가 B를 열었을 때, 유지가 정답일 사후 확률은 다음과 같습니다. (분모는 사회자가 B를 열 전체 확률인 $\frac{1}{2}$ 입니다.)

$$P(H_{keep}|E) = \frac{P(E|H_{keep}) \times P(H_{keep})}{P(E)} = \frac{\frac{1}{2} \times \frac{1}{3}}{\frac{1}{2}} = \frac{1}{3}$$

◆ 베이즈 정리

● 몬티홀 문제와 베이즈 정리

2. '선택 변경' 전략의 베이즈 정리

- 가설 (H_{switch}): 선택을 다른 문(C)으로 변경하는 것이 정답이다 (즉, 차가 C에 있다).
- 증거 (E): 사회자가 B 문을 열어 염소를 보여주었다.

1) 사전 확률 $P(H_{switch})$

처음에 차가 C에 있을 확률이므로 $\frac{1}{3}$ 입니다.

2) 우도 $P(E|H_{switch})$

만약 변경하는 것이 정답(차가 C에 있음)이라면, 참가자가 A를 골랐고 차는 C에 있으므로, 사회자는 차가 없는 B 문을 무조건(확률 1) 열어야만 합니다.

3) 사후 확률 계산

사회자가 B를 열었을 때, 변경이 정답일 사후 확률은 다음과 같습니다.

$$P(H_{switch}|E) = \frac{P(E|H_{switch}) \times P(H_{switch})}{P(E)} = \frac{1 \times \frac{1}{3}}{\frac{1}{2}} = \frac{2}{3}$$

◆ 베이지 정리

● 몬티홀 문제와 베이지 정리

전략 (가설)	사전 확률 (처음 가치)	우도 (조건에 따른 행동 확률)	사후 확률 (최종 승리 확률)
선택 유지 (H_{keep})	$\frac{1}{3}$	$\frac{1}{2}$ (B 또는 C 선택 가능)	$\frac{1}{3}$
선택 변경 (H_{switch})	$\frac{1}{3}$	1 (무조건 B만 열어야 함)	$\frac{2}{3}$

◆ 파이썬

- Thonny IDE 다운로드

<https://thonny.org>



The screenshot shows the Thonny website's download page. At the top, the word 'Thonny' is displayed in a large, bold, black serif font. To its right is a green download icon and the text 'Download version 5.0.0 for Windows • Mac • Linux'. Below this, a white box with a light gray border contains the heading 'Official downloads for Windows'. Under this heading, there are four download options, each with a description and a link. The third option, 'Portable variant with Python 3.14 for Intel or AMD computers, also usable on Arm', is highlighted with a red rectangular border. The fourth option is 'Re-using an existing Python installation (for advanced users)' with the command 'pip install thonny' below it. At the very bottom of the screenshot, a small portion of a code editor is visible, showing a function definition 'def fact(n):'.

Thonny

Download version [5.0.0](#) for Windows • Mac • Linux

Official downloads for Windows

Installer with Python 3.14 for Intel or AMD computers, requires Windows 10 or 11
[thonny-5.0.0-x64.exe \(24 MB\)](#) 🖱️ *recommended for you*

Installer with Python 3.14 for Arm (e.g. Snapdragon) computers
[thonny-5.0.0-arm64.exe \(23 MB\)](#)

Portable variant with Python 3.14 for Intel or AMD computers, also usable on Arm
[thonny-py38-5.0.0-windows-portable-x64.zip \(36 MB\)](#)

Re-using an existing Python installation (for advanced users)
pip install thonny

◆ 베이지 정리 코드

<https://colorscripter.com/>

```
# A: 충치, B: 치통

# P(A): 충치일 사전 확률,  $P(A) = P(A \text{ and } B) + P(A \text{ and } (\text{not } B))$ 
P_A = 0.04 + 0.06

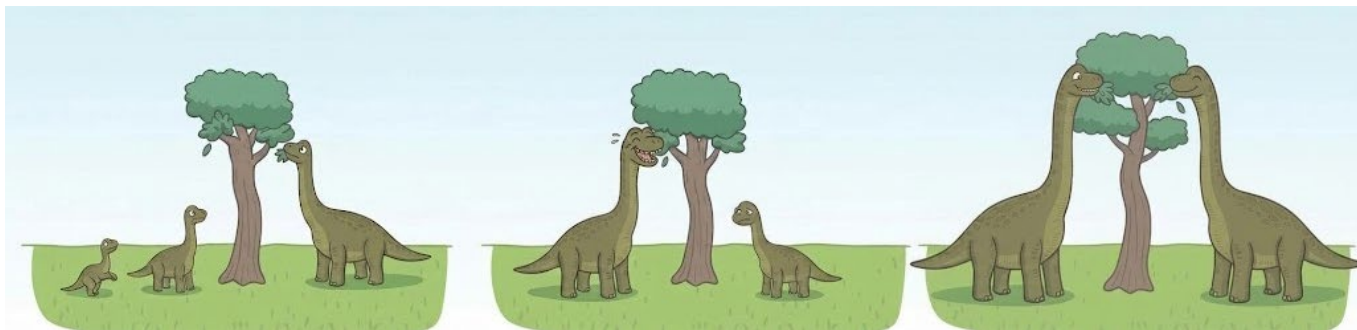
# P(B): 치통이 있을 사전 확률,  $P(B) = P(A \text{ and } B) + P((\text{not } A) \text{ and } B)$ 
P_B = 0.04 + 0.01

# P(B|A): 충치일 때 치통이 있을 확률, 가능도,  $P(B|A) = P(A \text{ and } B) / P(A)$ 
P_B_given_A = 0.04 / P_A

# 베이지 정리를 이용하여 사후 확률  $P(A/B)$  계산
P_A_given_B = P_B_given_A * P_A / P_B
print(P_A_given_B)
```

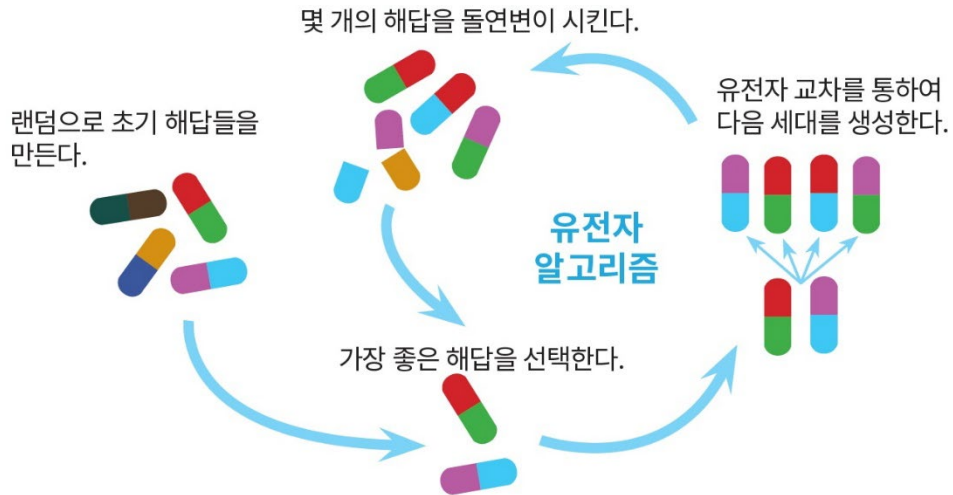
유전자 알고리즘

- ◆ 유전자 알고리즘은 생물체의 염색체가 유전되는 현상에서 영감을 얻은 최적화 알고리즘.
- ◆ 적자생존 원칙에 의해 교차, 돌연변이, 도태 등의 과정을 반영.
- ◆ 즉, 주어진 환경에 적합한 유전자만이 살아남는 자연계의 현상을 알고리즘으로 만든 것.



유전자 알고리즘

◆ 유전자 알고리즘 수행 과정



유전자 알고리즘

◆ 적합도(Fitness)

- 현재의 염색체가 얼마나 문제를 잘 해결하고 있는지에 대한 점수

◆ 평가함수(Evaluation Function)

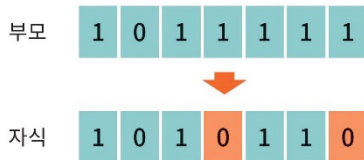
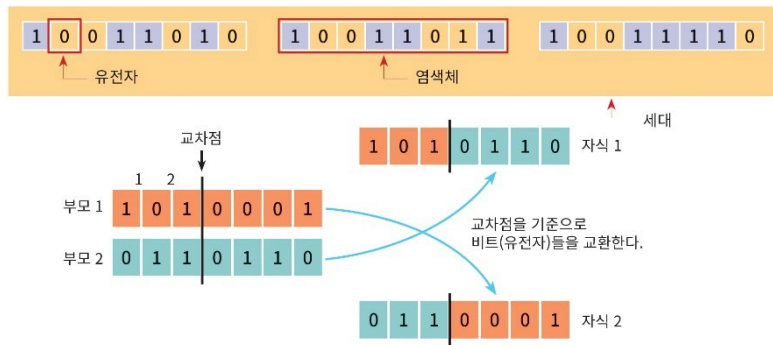
- 적합도를 계산하는 함수

◆ 교차(Crossover)

- 부모 염색체의 일부가 교환되는 것

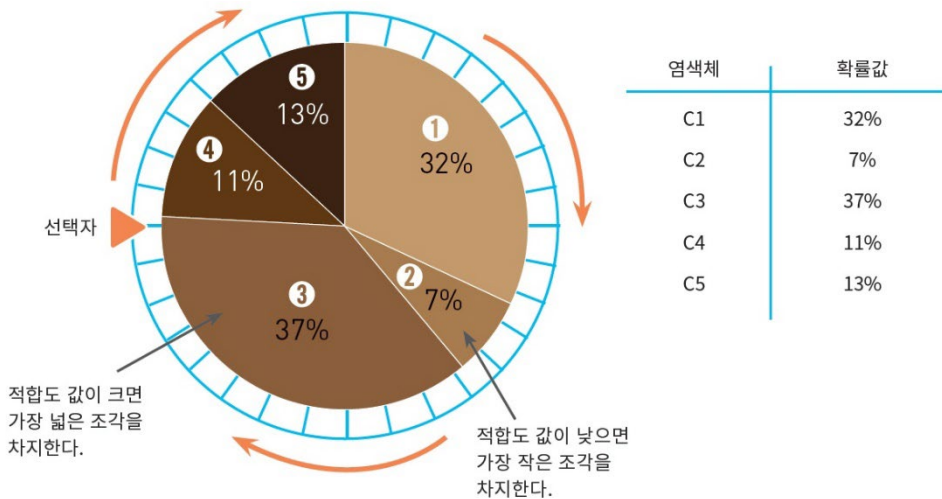
◆ 돌연변이(Mutation)

- 염색체에서 랜덤한 위치의 유전자가 값이 변동되는 것

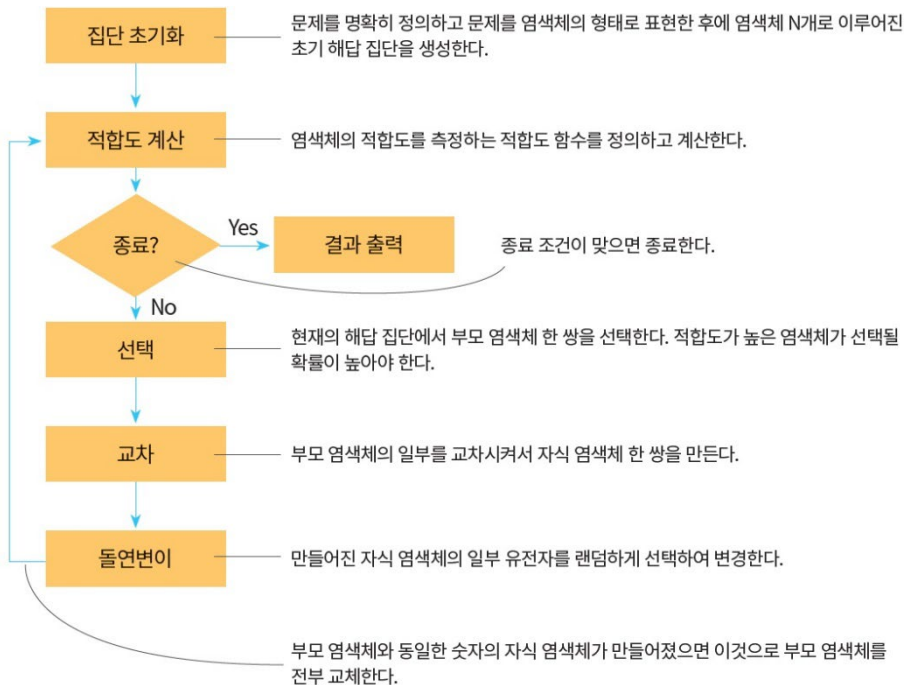


◆ 선택 연산자

- 유전자 알고리즘은 재산을 위하여 우수한 성능(높은 적합도)을 가지는 부모 염색체 2개를 선택
- 이때 사용되는 알고리즘이 룰렛 휠 선택 → 즉, 높은 적합도이면 높은 선택 확률

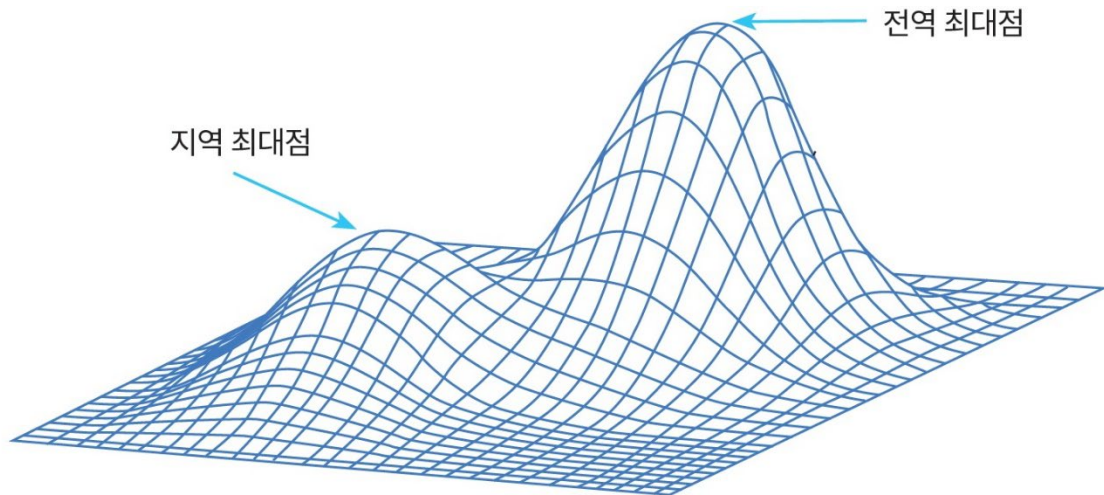


◆ 전체 과정



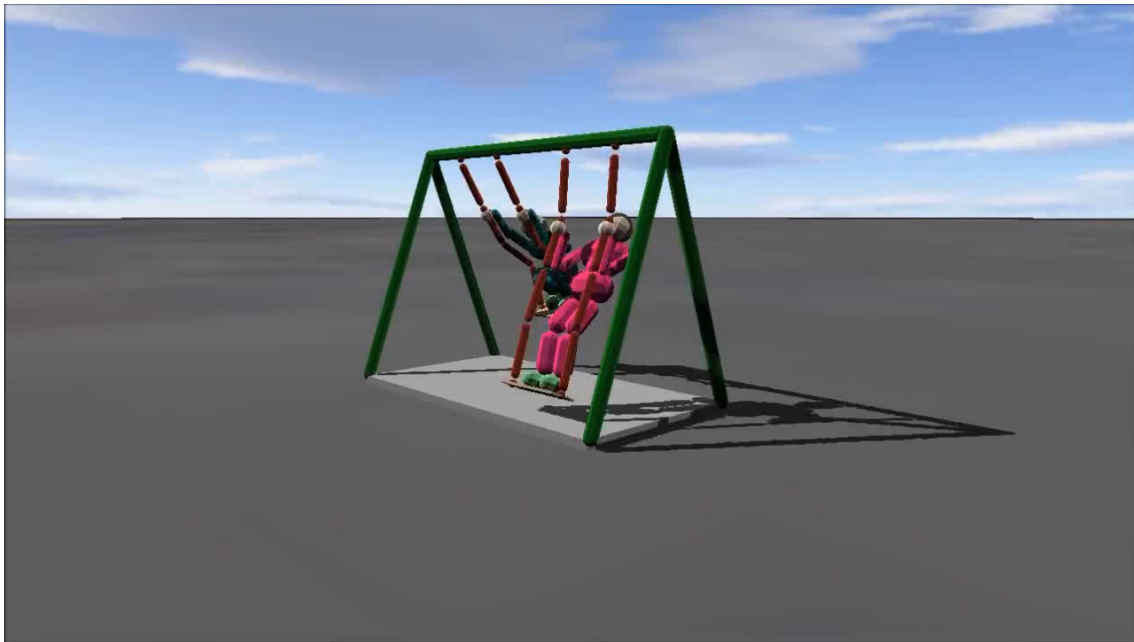
◆ 돌연변이 연산이 필요한 이유

- 지역 최대 문제에 빠지는 것을 방지



◆ 유전자 알고리즘 시뮬레이션

https://youtu.be/Yr_nRnqeDp0



◆ 장단점

- 결정론적 알고리즘이 존재하는 문제에 대해서는 유전자 알고리즘 접근법이 의미가 없다.
 - 즉, 명확한 답이 정해져 있는 문제에 적용하는 것은 무의미
- 유전자 알고리즘은 본질적으로 확률적 특성으로 인해 수행 시간을 예측하는 것이 어렵다.
- 만족 시켜야 하는 제약 조건이 많은 문제처럼 기존의 방법으로는 해결하기 어렵고 완벽한 최적 솔루션을 찾을 필요가 없는 문제에 주로 사용된다.

인공지능 코딩

◆ 유전자 알고리즘 코드

염색체(개체) 속 유전자는 1개

염색체(개체)는 10개

적합도: $f(x) = (x - 1)^2$

최소값 찾는 문제

선택: 개체 1순위, 2순위 선택

교차: 부모 유전자 평균

돌연변이: 유전자 값 임의로 변경

```
1 import random
2
3 # [목적] f(x) = (x-1)^2 의 최솟값 찾기 (정답 x = 1)
4
5 # 1. 설정값 (매우 단순화)
6 POP_SIZE = 10 # 개체수 10개
7 GENERATIONS = 30 # 30번 세대 교체
8 MUTATION_RATE = 0.3 # 30% 확률로 돌연변이 발생
9
10 # 임의의 초기 개체 생성 (-10.0 ~ 10.0 사이의 실수 10개)
11 population = [random.uniform(-10.0, 10.0) for _ in range(POP_SIZE)]
12
13 print("--- 초간단 유전자 알고리즘 시작 ---")
14
15 for generation in range(GENERATIONS):
16     # [평가 & 정렬] f(x) = x^2 값이 '작은 순서'대로 정렬합니다.
17     # 즉, 0번 인덱스(population[0])가 이번 세대에서 가장 우수한 정답(벨리트)이 됩니다.
18     population = sorted(population, key=lambda x: (x-1)**2)
19
20     # 현재 세대의 가장 우수한 해 출력
21     best_x = population[0]
22     print(f"[{generation + 1} 세대] 가장 우수한 x: {best_x:8.4f}, f(x): {(best_x-1)**2:8.4f}")
23
24     # 다음 세대를 저장할 리스트 (가장 똑똑한 상위 2개는 무조건 그대로 살려둠)
25     next_population = [population[0], population[1]]
26
27     # 나머지 8개의 자식을 채울 때까지 반복
28     while len(next_population) < POP_SIZE:
29         # [선택] 상위 4개(성적이 좋은 우등생들) 중에서 부모 2명을 무작위로 선택
30         parent1 = random.choice(population[:4])
31         parent2 = random.choice(population[:4])
32
33         # [교차] 두 부모의 중간값으로 자식을 만들 (부모의 성질을 닮음)
34         child = (parent1 + parent2) / 2
35
36         # [돌연변이] 일정 확률로 자식의 값을 살짝 비틀어줌 (+- 0.5 범위 내에서)
37         if random.random() < MUTATION_RATE:
38             child += random.uniform(-0.5, 0.5)
39
40         next_population.append(child)
41
42     # 세대 교체
43     population = next_population
44
45 # 최종 결과
46 final_best = population[0]
47 print("\n--- 최종 결과 ---")
48 print(f"찾아낸 최적의 x: {final_best:.4f}")
49
```

◆ 유전자 알고리즘 코드

```
import random
# [목적]  $f(x) = (x-1)^2$ 의 최솟값 찾기 (정답  $x = 1$ )
# 1. 설정값 (매우 단순화)
POP_SIZE = 10      # 개체수 10개
GENERATIONS = 30   # 30번 세대 교체
MUTATION_RATE = 0.3 # 30% 확률로 돌연변이 발생
# 임의의 초기 개체 생성 (-10.0 ~ 10.0 사이의 실수 10개)
population = [random.uniform(-10.0, 10.0) for _ in range(POP_SIZE)]
print("--- 초간단 유전자 알고리즘 시작 ---")
for generation in range(GENERATIONS):
    # [평가 & 정렬]  $f(x) = x^2$  값이 '작은 순서'대로 정렬합니다.
    # 즉, 0번 인덱스(population[0])가 이번 세대에서 가장 우수한 정답(엘리트)이 됩니다.
    population = sorted(population, key=lambda x: (x-1)**2)

    # 현재 세대의 가장 우수한 해 출력
    best_x = population[0]
    print(f"[{generation + 1} 세대] 가장 우수한 x: {best_x:8.4f}, f(x): {(best_x-1)**2:8.4f}")

    # 다음 세대를 저장할 리스트 (가장 똑똑한 상위 2개는 무조건 그대로 살려둬)
    next_population = [population[0], population[1]]

    # 나머지 8개의 자식을 채울 때까지 반복
    while len(next_population) < POP_SIZE:
        # [선택] 상위 4개(성적이 좋은 우등생들) 중에서 부모 2명을 무작위로 선택
        parent1 = random.choice(population[:4])
        parent2 = random.choice(population[:4])

        # [교차] 두 부모의 중간값으로 자식을 만들 (부모의 성질을 닮음)
        child = (parent1 + parent2) / 2

        # [돌연변이] 일정 확률로 자식의 값을 살짝 비틀어줌 (+/- 0.5 범위 내에서)
        if random.random() < MUTATION_RATE:
            child += random.uniform(-0.5, 0.5)

        next_population.append(child)

    # 세대 교체
    population = next_population
# 최종 결과
final_best = population[0]
print("\n--- 최종 결과 ---")
print(f"[찾아낸 최적의 x: {final_best:4f}]")
```